

Dum-E

A 157 B-parameter mixture of experts on a 16 GB laptop
Run report, specification changes, and the final configuration for the paper

Summary

Dum-E routes a Qwen2.5-0.5B gate over 100 Qwen2.5-1.5B LoRA experts into a Qwen3-4B synthesiser, entirely on one Apple M4 with 16 GB of unified memory. The architecture is ordinary; the discipline is not. Almost no quantity in the system is a number anyone chose. **k** comes from a RAM fit, the allocation law from a log-log regression that refuses itself when inadmissible, the reward from paired cross-entropy deltas rather than cosine self-agreement, the span bound from a measured backward-pass slope, and — as of this run — thermal regulation from 24 real die sensors rather than an operating-system ordinal that never moved.

The final run consumed **639,338 tokens** over **3,767 batches** at commit 0666188, with a clean working tree. It is the first run in the project's history in which the device cast a vote on k.

Headline results

Measure	Prior baseline (b2900)	Final run	Change
Tokens consumed	506,242	639,338	+26%
Batches	2,900	3,767	+30%
Clone fraction (experts saying the same thing)	0.25	0.01	25x lower
Routing confidence (rho)	0.326	0.613	+88%
Standing observations	7,001	9,129	+30%
Reliability observations	52,226	65,985	+26%
Expert updates applied	1,043	1,242	+19%
Negative-delta fraction	0.810	0.865	+7%
Thermal signal	1.0, constant	52.5-60.8 C	a sensor, at last
k_thermal distinct values	effectively 1	79	the device voted

The clone fraction is the one to read first. Experts are LoRA adapters over a single frozen base, so they begin mathematically identical and can only diverge by training. At 0.25 a quarter of batches had at least two experts emitting identical text; at 0.01 the pool has genuinely differentiated.

Every run

Nothing is deleted when a run ends. Logs are archived under `logs/archive/` and the whole of `state/dume/` is moved aside, because a half-cleaned run is neither fresh nor a continuation and there is no way to tell afterwards which it was.

Run	Batches	Tokens	Outcome
run_20260920_b3425	3,425	596,215	Archived for the paper. Predates the thermal regulator entirely.
run_20260921_b2900	2,900	506,242	Flat TARGET_MAX_TOKENS=128 baseline at 4eef0dc. Thermal level 1.0 on
aborted: fairshare	1,649	-	Abandoned mid-run on a token-division change.
aborted: klen	62	-	k-from-input-length; abandoned.

Run	Batches	Tokens	Outcome
aborted: thermreg	53	-	First thermal regulator attempt; abandoned.
aborted: b12 / b21 / b25	15 / 22 / 26	-	Three early cold-start failures.
archive 20260921_1856	3,156	506,242	The 1M-token supervisor line.
archive 20260921_2145	542	94,641	500k run on caf80b1; stopped deliberately to fix the thermal sensor.
archive 20260921_2242	216	37,292	Restarted on 6e2adda; stopped again to add the run-fraction scaling.
run_20260922_final	3,767	639,338	THE PAPER RUN. Commit 0666188, clean tree, rc=0.

The final run overshot its 500,000-token target to 639,338 because the supervisor sized its cycle from a 146 tokens-per-batch estimate and the run actually averaged 170. Overshoot is harmless; the target is a floor.

What was added

A real thermometer

`models.die_temp()` reads 24 SoC die sensors through IOKit's HID event system: match on HID usage page 0xff00 / usage 5, then `IOHIDServiceClientCopyEvent` with `kIOHIDEventTypeTemperature`. No sudo, no subprocess, no new package — pure ctypes against the IOKit framework. Measured cost: 16 ms for all 24 sensors, 0.03% of a 12.9 s batch.

The **mean** of the die sensors, not the max. Over a 60 s trace while training, `max(tdie)` had standard deviation 1.29 C and 4.08 C of range against `mean(tdie)`'s 0.36 and 1.13. The max is a per-core spike detector and at one read per batch it samples noise. Battery temperature is also sudo-free and was rejected on measurement: 31.19 to 31.20 C over five minutes at full load.

A regulator with four moving parts

```
span      = peak - floor           the range this die actually works over
z         = (T - mean) / span      how far above its own normal it sits now
pressure  = max(0, z - mean|z|)    less the jitter it shows while idle
k         = k_max ** (1 / (1 + pressure * left))
                                                unless the OS reports `serious`, then k = 1
```

Every quantity is measured on the machine. The two that are not ours belong to Apple (`NSProcessInfoThermalStateSerious`) and to the RAM fit (`k_max`).

The run-fraction scaling

`left` is the fraction of the run still to do, and it **multiplies** the heat term rather than adding to it. Heat is a forecast — this machine will be in trouble if it keeps working like this — and how much that matters depends entirely on how much working is left. A die climbing over the last hundred batches has nowhere to get to before the run ends.

left	1.00	0.75	0.50	0.25	0.10	0.00
k when hot	2.712	2.924	3.191	3.537	3.797	4.000
k when cool	4.000	4.000	4.000	4.000	4.000	4.000

Multiplying is also what settled a problem that adding could not. A pressure in span-fractions and a time in seconds have no common unit, and every attempt to find one either cancelled — `seconds_left` is itself built from `k`, so the ratio collapses to `1/k` — or pinned `k` at 1 for most of a long run. A multiplier needs no common unit.

A tool that asks whether a mechanism ever fired

`scripts/did_it_fire.py` reads a run's health records and asks, per field, whether the number ever changed. Two things make it more than a diff.

The tail. Movement alone is not the test. In the run that hid the thermal bug, `k_thermal` did move — 2.758 to 4.000 during warmup, as its baseline converged onto the one level it would ever see — and then never again for 2,900 batches. Every field is judged on the last half of the run as well as the whole of it.

The chain. A constant output is not by itself a bug: a cool machine should produce no thermal pressure. It is a bug when the output is constant while its input moved. When the input is dead too, the fault is upstream at the sensor — which is where it actually was.

What was removed

Every deletion below was made because a measurement said the thing was not doing work, not because it looked complicated.

Removed	Why, with the number
The whole ordinal apparatus: baseline, volatility, gap_up, gap_down, excess, the k ramp, the learned threshold	All of it existed to squeeze a rate out of a four-valued ordinal that has no usable pointwise derivative. With rea
Explicit first and second derivatives of temperature	Built, measured, cut. On the raw reading they are noise — differencing doubles it each time, and a SETTLED
The k_time term (commit 7c60d04)	$\tau / (a + b \cdot \text{alloc}(T))$, with tau the mean of the same latency bank the line (a,b) was fitted to. One curve divided
A second-order k with its own velocity and two thresholds	Built across four commits, then reverted whole on the author's call. Parked on branch thermal-second-order a
Dead tier / _settled plumbing (commit 2f86048)	Reward reduced to a plain mean over tokens once the tier machinery was shown to have no consumer.

Why the derivatives were redundant rather than merely weak

z is already the rate term. Because the mean lags, a die that has just reached 60 C scores $z = +0.504$ and one that has been at 60 C for 400 reads scores $z = 0.000$. A fast reading against a slowly re-learning normal is a high-pass filter whose time constant is the machine's own history. The second-order behaviour lives in the gap between the two, not in a difference taken on either.

Every problem, and how it was fixed

Each entry below follows the same shape, because that is how each was actually found: a symptom, a measurement that made it undeniable, and a fix whose guard fails when the fix is reverted. Every guard named here was mutation-tested — the mechanism was deliberately broken and the assertion confirmed to fire.

1. The thermal sensor was a constant

Symptom. The regulator was correct in every line, logged clean health records for thousands of batches, passed every assertion in the check harness, and never once moved `k`.

Measurement. `NSProcessInfo.thermalState()` returned `fair` on all 580 samples of a 2,900-batch run, all 541 of the next, and on five direct polls under sustained load. Downstream: `thermal_base` converged 0.7 to 1.0, `thermal_vol` sat at 0.000-0.005, `gap_up` and `gap_down` were never set because no step ever happened, and `k_thermal` went 2.758 to 4.000 and stayed. The machine genuinely sits at `fair`; the die underneath swings 52 to 67 C in a minute.

Fix. Replaced the ordinal with mean die temperature from 24 IOKit HID sensors. The ordinal is retained, but only as a veto at `serious`. The old docstring claiming degrees need root was wrong — it cited powermetrics and never tried the HID path every Mac monitor uses.

Guard: `die_temp()` must return a plausible temperature; the check fails if the sensors are absent.

2. `k_thermal` was read twice per batch

Symptom. `Scheduler.k_thermal` is a property with a side effect — it advances the regulator — and it was read by the router and again by the health record.

Measurement. Every interval the regulator measured was therefore in half-batches, and the health record logged a different `k` than the router had actually used. Confirmed live after the fix: 25 regulator reads at batch 25, where it would previously have been 50.

Fix. The property caches into `last_k_thermal`, which is what reporting reads.

Guard: A source check asserts exactly one caller outside the scheduler — and strips comments first, because the first version of that guard matched a comment that merely mentioned the name.

3. The health record was never cleared

Symptom. `Health.rec` was built in `__init__` and never emptied, so any conditionally-written field kept its last value forever and every later record repeated it as though freshly measured.

Measurement. Over the final run's 753 records: `graded` repeated on 100% of consecutive records, `imitate_loss` on 44%, `expert_loss` on 16%, `alloc_width_var` on 95%. The loss curves plotted smoother than the run actually was.

Fix. `rec` is cleared after each emit, so a field not measured in a window is absent rather than stale. The argument for this was *already in the same function*, written four lines below for `self.alarms`: "a record whose job is to say whether a mechanism is doing anything must not report history in the present tense." It had been applied to one path and not the other.

Guard: A conditional field written in one window must be absent from the next.

4. ``graded`` could not report a refusal

Symptom. The field named for whether a batch was graded was 1.0 on every single record of the run.

Measurement. `rec["graded"] = 1.0` was assigned only inside the admit branch, so a refusal did not write the key at all and the previous 1.0 stood. The batch lines say what the record could not: **2381 ADMIT, 945 HELDOUT, 441 REFUSE**. 11.7% of batches were refused and the health record showed none of it.

Fix. `graded` moved into the base record at 0.0, overwritten to 1.0 on admit.

Guard: A source check asserts it is written unconditionally.

5. imitate_loss had no honest source

Symptom. It was recorded into the health record and never printed per batch, so the stale-carrying record was its only source.

Measurement. 44% of its points were carried forward. Unlike gate, expert and central loss — which the per-batch lines print only when computed, at 0.4%, 2.0% and 2.2% repeats respectively — it cannot be reconstructed from the finished run.

Fix. It now prints on the batch line alongside the others.

Guard: A general rule: every `rec["*_loss"]` in `train.py` must appear in the batch line.

6. A guard whose fixture proved nothing

Symptom. A guard was written to assert the jitter deadband cannot lock itself shut. Mutating the deadband to freeze after warmup — precisely the failure it named — passed every check in the file.

Measurement. The fixture warmed up with `np.linspace`, and a perfectly smooth monotone ramp is the one input that never inflates the deadband: `run` accumulates every read, the mean tracks the ramp exactly, `z` is identically zero and `dev` stays at **0.0000** for the whole warmup — against 0.158-0.258 measured on the real machine.

Fix. The fixture now jumps and holds, as the real die does, reproducing `dev` 0.32 to 0.19 to 0.12 over the first 50 reads. The freeze mutation is now caught, by this guard and by nothing else.

Guard: The guard asserts on its own fixture first: if the warmup does not inflate the deadband past 0.1, it fails and says so rather than passing while testing nothing.

7. The run horizon reached Python only by inheritance

Symptom. `TARGET` was set as a shell variable in the supervisor and reached the training process only because a parent script happened to export it.

Measurement. A bare invocation of the supervisor would have left it invisible to Python, making `left` 1.0 for the entire run — the scaling silently dead, with nothing to indicate it.

Fix. The supervisor exports it. An undeclared horizon yields `left = 1.0`, which is the full heat response: the safe direction to be wrong in.

Guard: `left` is clamped to `[0, 1]` on the way in, so it can only reduce the heat response, never amplify it.

A note on the pattern

Problems 1, 3, 4, 5 and 6 are the same bug wearing different clothes: **something that runs perfectly and reports nothing**. A sensor that never moves, a record that reports history as the present, a field that cannot express its own negative case, a curve with no honest source, and a test fixture that never exercises the thing it tests. None of them is a crash, none produces a wrong number that looks wrong, and none is findable by reading the code — every one was found by asking a log whether a value had ever changed.

Problem 6 is the sharpest of them, because it happened *while fixing* problem 1, in the same file, on the same day. That is the argument for `did_it_fire.py` and for the fixture-sanity assertion existing as permanent tooling rather than as a one-off diagnosis.

Final run configuration

This is the configuration of run_20260922_final, the run the paper reports. Commit 0666188, branch symbiosis-curriculum, working tree clean (logs/DIRTY empty, so it is reproducible from the commit).

Models

Role	Model	d_model	Notes
Gate	Qwen2.5-0.5B-Instruct-4bit	896	Routes and writes the shared map
Expert x100	Qwen2.5-1.5B-Instruct-4bit	1536	LoRA r=8, alpha=16, dropout=0.05, over ONE frozen base
Central	Qwen3-4B-Instruct-2507-4bit	2560	Synthesiser; inherits a grounded-CE checkpoint

Nominally ~157 B parameters. It fits in 16 GB because the 100 experts are LoRA adapters over a single frozen 4-bit base — 35 MB each, not 1.5 B weights each.

Pool geometry (all derived, none chosen)

E = 100 experts
GENERAL_EXPERTS = ceil(sqrt(E)) = 10
CENTROID_EXPERTS = floor(sqrt(90)) = 9
MAX_CLUSTERS = 90 // 9 = 10
k_max = min(k_fit, k_tier, MAX_CLUSTERS) = 4 on this machine
k_tier = floor(sqrt(RAM_GB)) = 4
space for k = R - sqrt(R) - central - gate (16384 - 4096 - 2160 - 268 MB)

Training parameters

LR	2e-05	GRAD_CLIP	1.0
LORA_R / ALPHA / DROPOUT	8 / 16 / 0.05	SAMPLE_TEMP	0.8
EXPERT_GEN_TOKENS	32	TARGET_MAX_TOKENS	128
TARGET_MIN_TOKENS	4	EXPERT_PROBE_TOKENS	256
HEALTH_EVERY	5	SAVE_EVERY	25
MIGRATE_EVERY	20	HELDOUT_MOD	4
MIN_MEMBERS	10	LOAD_WINDOW_PER_CLUSTER	10
SIM_MEMBER / NEIGHBOUR / FAR	0.90 / 0.70 / 0.40	TAU_MAX / PERCENTILE / STEP	0.97 / 10 / 0.005
RELIABILITY_MIN_OBS	100	STANDING_A_WINDOW	20
R_MIN	0.05	CHAIN_MEMORY / PRIOR	500.0 / 1.0
RUN_TARGET_TOKENS	500,000	state.VERSION	7

Thermal regulation

sensor mean of 24 tdie sensors, IOKit HID usage page 0xff00 / usage 5
cost 16 ms per read, one read per batch
span peak - floor, the measured working range
z (T - mean) / span
mean re-learns: mean += min(1, run/n) * (T - mean)
pressure max(0, z - mean|z|)
k k_max ** (1 / (1 + pressure * left))
veto NSProcessInfoThermalStateSerious (=2) -> k = 1, absolute
left 1 - clock / RUN_TARGET_TOKENS, clamped to [0, 1]

What the run measured

Quantity	Value
Tokens / batches	639,338 / 3,767
Decisions	2381 ADMIT, 945 HELDOUT, 441 REFUSE
Wall clock	~10.5 h, 13.2 tokens/s consumed, 170 tokens/batch
Die temperature	52.5 - 60.8 C, mean settled at 54.9 C
OS ordinal	1 distinct value across all 753 records (`fair`, constant)
Thermal pressure	fired on 15.7% of reads
k_thermal	3.422 - 4.000, 79 distinct values
k actually run	4 on 685 records, 3 on 48, 2 on 20
Timeline A (Central alone)	29.9% of inputs
gate_loss	0.3868 -> 0.1545 (2381 points)
central_loss	1.2801 -> 0.3147 (2822 points)
expert_loss	0.0006 -> 0.0229 (1243 points)
Clone fraction	0.01 (from 0.25 on the prior baseline)
Update fraction	0.55
Standing / reliability observations	9,129 / 65,985

Read the paper's loss curves off the per-batch log lines, not off health.csv. The health record carried conditional fields forward (see problem 3); the batch lines print each loss only when it was computed, and there are five times as many of them.

The isolated effect of the run-fraction scaling

Pressure is logged before scaling, so the raw fall in pressure over the run is the regulator settling, not the scaling. Holding pressure fixed isolates the scaling's own contribution:

left band	n	mean pressure	k if left=1	k actual	experts kept
1.00 - 0.75	43	0.0426	3.790	3.823	+0.033
0.75 - 0.50	22	0.0267	3.862	3.905	+0.043
0.50 - 0.25	7	0.0167	3.911	3.961	+0.050
0.25 - 0.00	46	0.0272	3.860	3.984	+0.124

Monotone across the whole run. The last band is the clearest: the die warmed again near the end to a pressure comparable with the first band, and the scaling kept +0.124 experts instead of +0.033. Same heat, four times the leniency, because there was almost no run left to protect.

Verification

Two independent checks, both runnable and both part of the repository.

scripts/dume_check.py

Assertion harness over every mechanism. Each thermal assertion in it was mutation-tested: the mechanism was deliberately reverted and the assertion confirmed to fire. Mutations checked — removing the jitter deadband, scaling by the noise floor instead of the span, dropping the OS veto, freezing the mean at 1/n, removing the floor at zero, restoring the double read, making left add instead of scale, ignoring it, unclamping it, freezing the deadband after warmup, moving graded back to the admit path, and removing a loss from the batch line. Each fails a different, named assertion.

scripts/did_it_fire.py

Run against the final run: **753 records, 0 problems, all six input-to-output chains ok**. Run against the archived logs of the run that hid the thermal bug, it prints the whole diagnosis, including that the fault was at the sensor:

```
thermal      108    1    1.0000    1.0000    CONSTANT
thermal_gap_up 108    1    7.0000    7.0000    CONSTANT
thermal_excess 108    1    0.0000    0.0000    CONSTANT
k_thermal     108    2    3.3040    4.0000    WARMUP ONLY
thermal_p -> k_thermal  input dead too -- look UPSTREAM of thermal_p
```

Three fields are still constant in the final run, and each correctly so: thermal_floor and thermal_peak are running extremes, so constant means the die never left its established range; thermal_level is the OS ordinal, still pinned at fair for all 639,338 tokens — the original diagnosis holding true live, while the die beside it swings 8 C and moves k.

Known and deliberately deferred

Item	Status
Expert-pass amortisation	A pass costs a = 0.542 s fixed + b = 0.00507 s/token and experts write 8-29 tokens, so ~84% of a pass is fixed cost
imitate_loss for this run	44% carried forward, no per-batch source. Fixed going forward; not reconstructable from this run. A re-run is the only way to get it
Per-expert training budget	~151 selections per expert across the run. This is a mechanism test, not a capability run; whether expert identity ever changes is unknown
MIGRATE_EVERY = 20	Suspected thrashing; unexamined.
state.VERSION bump	Silently discards clock, batch and consumed.